CSCI 1101 Programming Principles I · Project 1: Simplified Blackjack

- **Section:** Abbas Aliyev's section, Fall 2026
- **Type:** individual programming project
- **Weight:** 10% of the course grade
- **Announced:** Friday 25 September 2026
- **Deadline: Sunday 11 October 2026, 23:59** (Baku time), on Blackboard

The goal. You write a Blackjack card game in C that runs in the terminal. You play against the computer, which plays the dealer. The project puts together what you learned in the first weeks: variables, `printf` and `scanf`, `if` and `else`, loops, random numbers, and, from Tuesday 29 September, **functions**.

1. What the game is

In Blackjack you collect cards. Each card is worth some points. You want a total **as close to 21 as possible, but not above 21**. You play first. Then the dealer plays. Then the two totals are compared.

This is **not** the full casino game. Some rules are made simpler on purpose, so that you can focus on the programming.

Important: if you already know Blackjack, forget the casino rules. Follow the rules in this document. Your program is graded on these rules.

2. What you will learn

When you finish this project, you will be able to:

- keep the state of a game in variables and change it step by step;
- make random numbers and use them;
- make decisions with `if` and `else`;
- repeat actions with `while` and `for`;
- check what the user types and ask again when it is wrong;
- split one program into small functions, each with one job;
- explain how your own program works, and change a small part of it when you are asked.

3. The rules of the game

3.1 Cards and their points

There are no suits (no hearts, no spades). Only the card's name counts.

Card	Shown as	Points
Ace	A	1
Number cards	2, 3, 4, 5, 6, 7, 8, 9, 10	the number itself
Jack	J	10
Queen	Q	10
King	K	10

The Ace is always worth 1. It never becomes 11. So A and K together make 11, not 21.

Every time a card is drawn, it is one of these 13 cards, chosen at random: A, 2, 3, 4, 5, 6, 7, 8, 9, 10, J, Q, K.

There is no deck of 52 cards. You do not keep track of which cards were already drawn. Every draw is new, so the same card can come many times in one game.

3.2 Your turn (the player)

- You always play first. Your total starts at 0.
- You get one card at a time.
- After every card, the program shows the card and your new total.
- If your total is **above 21**, you **bust**: you lose at once, and the game ends.
- If your total is 21 or less, the program asks you: **Hit** or **Stand**?
  - **Hit** means: give me another card.
  - **Stand** means: I stop. Now the dealer plays.

Here is a turn where the player hits once and then stands. The numbers after `Choice:` are what the player typed.

```
PLAYER TURN
You draw: 7
Your total: 7
1. Hit
2. Stand
Choice: 1
You draw: K
Your total: 17
1. Hit
2. Stand
Choice: 2
```

3.3 The dealer's turn

- The dealer plays **only after you stand**. If you bust, the dealer does not play at all.
- The dealer's total starts at 0.
- The dealer asks nothing. The program plays for the dealer.
- The dealer takes one card at a time **while the dealer's total is less than 17**.
- As soon as the dealer has 17 or more, the dealer stops (stands).
- If the dealer's total goes **above 21**, the dealer busts and **you win**.

So: at 16 the dealer takes another card. At 17 the dealer stops.

```
DEALER TURN
Dealer draws: 9
Dealer total: 9
Dealer draws: 6
Dealer total: 15
Dealer draws: 3
Dealer total: 18
Dealer stands.
```

3.4 Who wins

Check these rules in this order:

1. If you go above 21, you lose at once. The dealer does not play.
2. If you stand, and then the dealer goes above 21, you win.
3. If nobody went above 21, compare the two totals.
4. The higher total wins.
5. If the totals are equal, the game is a **draw**.

3.5 Play again

After every game, the program asks:

```
1. Play again
2. Quit
Choice:
```

If you type 1, a new game starts: both totals start again from 0. If you type 2, the program prints how many games you won, lost and drew, and ends.

4. The order of one game

Your program should go through these steps:

1. Start the program. Set up random numbers (section 6.2) and the three counters: wins, losses, draws.
2. Start the player's turn with a total of 0.
3. Draw one card for the player and show it.
4. Add its points to the player's total and show the total.
5. If the player's total is above 21, say so, the dealer wins, and the game ends.
6. Otherwise, ask: Hit or Stand?
7. Go back to step 3 while the player chooses Hit.
8. When the player chooses Stand, start the dealer's turn with a total of 0.
9. The dealer draws cards while the dealer's total is less than 17.
10. If the dealer's total is above 21, say that the dealer busts and the player wins.
11. Otherwise, show both totals and say who won, or that it is a draw.
12. Add 1 to the right counter (wins, losses or draws).
13. Ask: Play again or Quit? Go back to step 2 on Play again.
14. On Quit, print the three counters and end.

5. Example runs

These runs were made with the course's own solution. Your program must show the **same information** in the same order. The exact words and empty lines may be different. The numbers after `Choice:` are what the player typed.

Example A: you stand and win

```
=== SIMPLIFIED BLACKJACK ===

PLAYER TURN
You draw: 8
Your total: 8
1. Hit
2. Stand
Choice: 1
You draw: Q
Your total: 18
1. Hit
2. Stand
Choice: 2

DEALER TURN
Dealer draws: 10
Dealer total: 10
Dealer draws: 7
Dealer total: 17
Dealer stands.
```

```
Player total: 18
Dealer total: 17
You win!

1. Play again
2. Quit
Choice: 2

Wins: 1, losses: 0, draws: 0
Goodbye!
```

Example B: you bust

The game ends at once. The dealer does not play.

```
PLAYER TURN
You draw: K
Your total: 10
1. Hit
2. Stand
Choice: 1
You draw: 8
Your total: 18
1. Hit
2. Stand
Choice: 1
You draw: 6
Your total: 24
BUST! Your total is above 21.
Dealer wins.
```

Then `1. Play again`, `2. Quit`, as in Example A.

Example C: the dealer busts

At 16 the dealer must take another card.

```
PLAYER TURN
You draw: 10
Your total: 10
1. Hit
2. Stand
Choice: 1
You draw: 9
Your total: 19
1. Hit
2. Stand
Choice: 2

DEALER TURN
Dealer draws: K
Dealer total: 10
Dealer draws: 6
Dealer total: 16
Dealer draws: 8
Dealer total: 24
Dealer busts!
```

```
You win!
```

Example D: a draw

```
PLAYER TURN
You draw: 10
Your total: 10
1. Hit
2. Stand
Choice: 1
You draw: 8
Your total: 18
1. Hit
2. Stand
Choice: 2

DEALER TURN
Dealer draws: 9
Dealer total: 9
Dealer draws: 9
Dealer total: 18
Dealer stands.

Player total: 18
Dealer total: 18
Draw!
```

Example E: the dealer wins

```
PLAYER TURN
You draw: 7
Your total: 7
1. Hit
2. Stand
Choice: 1
You draw: K
Your total: 17
1. Hit
2. Stand
Choice: 2

DEALER TURN
Dealer draws: 9
Dealer total: 9
Dealer draws: 6
Dealer total: 15
Dealer draws: 3
Dealer total: 18
Dealer stands.

Player total: 17
Dealer total: 18
Dealer wins.
```

Example F: a wrong choice, then two games

The player types `3` by mistake and is asked again. After the first game the player plays again, and then quits.

```
=== SIMPLIFIED BLACKJACK ===

PLAYER TURN
You draw: K
Your total: 10
1. Hit
2. Stand
Choice: 1
You draw: K
Your total: 20
1. Hit
2. Stand
Choice: 3
Please type 1 or 2.
Choice: 2

DEALER TURN
Dealer draws: 6
Dealer total: 6
Dealer draws: J
Dealer total: 16
Dealer draws: Q
Dealer total: 26
Dealer busts!
You win!

1. Play again
2. Quit
Choice: 1

PLAYER TURN
You draw: 8
Your total: 8
1. Hit
2. Stand
Choice: 1
You draw: K
Your total: 18
1. Hit
2. Stand
Choice: 2

DEALER TURN
Dealer draws: A
Dealer total: 1
Dealer draws: 9
Dealer total: 10
Dealer draws: Q
Dealer total: 20
Dealer stands.

Player total: 18
Dealer total: 20
Dealer wins.
```

```
1. Play again
2. Quit
Choice: 2

Wins: 1, losses: 1, draws: 0
Goodbye!
```

Your cards will be different every time you run your program: they are random.

6. What your program must do

6.1 Requirements

- It is written in C and runs in the terminal.
- Every card is chosen at random from the 13 cards A to K.
- The card shown matches the points added: A adds 1; J, Q and K add 10.
- The player can choose Hit or Stand again and again.
- The player loses at once when the total goes above 21.
- The dealer plays only after the player stands.
- The dealer draws while the total is less than 17, and stops at 17 or more.
- The program finds every result correctly: player bust, dealer bust, player wins, dealer wins, draw.
- After each game it asks Play again or Quit, and at the end it prints the numbers of wins, losses and draws.
- The screen shows enough for the player to follow the game: each card, each total, the choices, the result.
- The code is easy to read: clear variable names, and the lines inside `{ }` moved to the right (indented).
- The program is split into functions (section 6.4), not written all inside `main`.

6.2 Random cards: copy these lines

You have not learned random numbers in class, so here is exactly what to write.

At the top of the file, under `#include <stdio.h>`, add two more lines:

```
#include <stdlib.h>
#include <time.h>
```

`stdlib.h` gives you `rand` and `srand`. `time.h` gives you `time`.

As the **first line** inside `main`, write this line once:

```
srand(time(NULL));
```

This starts the random numbers from the clock, so every run of the program gets different cards. Copy it as it is; the `NULL` inside is explained later in the course.

To get one random card from 1 to 13, write:

```
int card = rand() % 13 + 1;
```

`rand()` gives a big random whole number. `% 13` keeps the remainder after dividing by 13, which is 0 to 12. `+ 1` moves it to 1 to 13.

Call `srand` **only once**, at the start of `main`. Do not put it inside a loop or inside the function that draws a card: then you get the same card many times in a row.

6.3 Numbers for cards

Store a card as a whole number from 1 to 13. Then decide how to show it and how many points it is worth:

Number	Shown as	Points
1	A	1
2 to 10	the same number	the same number
11	J	10
12	Q	10
13	K	10

This table tells you how to store a card. It is not the whole program: the logic is yours to write.

6.4 The functions to write

Functions come in class on **Tuesday 29 September** (class 7). Until then, you can start with everything in `main`. After Tuesday, move each job into its own function. Write at least these functions, with these first lines. You may add more functions.

Function	What it does
`int draw_card(void)`	Returns a random card, a number from 1 to 13.
`int card_points(int card)`	Returns the points of a card: 1 for A, the number for 2 to 10, 10 for J, Q and K.
`void print_card(int card)`	Prints the card as A, 2 ... 10, J, Q or K. It returns nothing.
`int read_choice(void)`	Prints `Choice: `, reads a number, and asks again until it is 1 or 2. Returns it. Use it for both menus.
`int player_turn(void)`	Plays the whole player's turn. Returns the player's final total (above 21 means bust).
`int dealer_turn(void)`	Plays the whole dealer's turn. Returns the dealer's final total.
`int play_game(void)`	Plays one whole game with the two functions above and prints the result. Returns 1 if the player won, -1 if the dealer won, 0 for a draw.

Then `main` becomes short: `srand` once, a loop that calls `play_game`, adds to the right counter and asks Play again or Quit, and at the end the three counters.

6.5 What the user types

Both menus use numbers:

```
1. Hit
2. Stand
Choice:
```

If the user types a number that is not 1 or 2, it is not a move. Say so and ask again, until the user types 1 or 2. You saw this in class 6: asking again with `while`. You only need to handle numbers. If a user types a letter, your program does not have to cope with it.

7. What you do not need to do

These are **not** part of Project 1:

- card suits;
- a real deck of 52 cards;
- removing a card from the deck after it is drawn;
- shuffling a deck;
- a hidden dealer card;
- two cards at the start for the player or the dealer;
- an Ace worth 11;

- a special "Blackjack" for 21 with the first cards;
- bets, chips, money, insurance, splitting or doubling down;
- windows, buttons or pictures.

Do not make it more complicated. Extra casino rules do not replace any required part. Getting the simple rules right comes first.

8. Special cases your program must get right

Situation	What must happen
The player reaches exactly 21	The player is not bust. The player may stand. The menu still offers Hit; if the player hits, that is the player's choice.
The player goes above 21	The game ends at once. The dealer does not play.
The dealer reaches exactly 17	The dealer stops.
The dealer has 16	The dealer must take another card.
The dealer reaches 21	The dealer stops. Compare the totals as usual.
The dealer goes above 21	The dealer busts and the player wins.
The totals are equal	The game is a draw.
An Ace is drawn	Add 1. Always.
J, Q or K is drawn	Add 10. Always.
The user types 3, 0 or -1 at a menu	Not a move. Ask again.
The player chooses Play again	A new game: both totals start from 0. The counters keep their values.

9. How to hand it in

- Hand in **one file**, named `blackjack.c`: the C source file. Not the `.exe`, not a zip, not a screenshot, not a Word file.
- Upload it on Blackboard, in the assignment **Project 1: Simplified Blackjack**, in the folder *Projects* in Course Content. You may upload up to 3 times; only your last upload is graded.
- **Deadline: Sunday 11 October 2026, 23:59** (Baku time).
- Before you upload, compile it once more with `gcc -Wall blackjack.c -o blackjack` and play a few games. A file that does not compile cannot show that it works.
- Upload early. The syllabus rule on late work: "Deadlines will never be extended. When submitting, a student must consider possible internet connection issues, uploaded file size, etc. Late submissions of one minute until thirty minutes will receive a 10% penalty, late submissions of thirty minutes until one day will receive a 25% penalty with no claim for bonuses (if there are any). For each next day, a further 25% penalty will be applied."

10. Your own work, and explaining it

This is an **individual** project. You may talk with classmates about general ideas ("how does a `while` loop that asks again work?"). The code you hand in must be written by you. Do not copy another student's code, and do not hand in code that you cannot explain.

From the syllabus: "Each student is responsible for understanding all parts of the code submitted as part of a project. After submission, students may be asked by the Teaching Assistant to explain selected parts of their implementation, describe how particular functions or algorithms work, or make a small modification to their program. This verification is considered part of the project assessment. A student who is unable to demonstrate sufficient understanding of the submitted work may receive a reduced project grade."

11. How it is graded

Out of 100 points.

Area	Points	What is checked
Game correctness	45	Cards and points (10). The player's turn: Hit and Stand, and a bust ends the game at once (10). The dealer's turn: starts only after Stand, draws below 17, stops at 17 or more (10). The result: player bust, dealer bust, higher total, draw (15).
Control flow	15	Loops for the two turns (5). Asking again after a wrong choice (5). Play again, with the counters kept (5).
Program organisation	15	The functions of section 6.4, each doing its one job; a short `main`.
Code quality	10	Clear names, indentation, a few comments where they help, no warnings from `gcc -Wall`.
Input and output	15	The screen shows each card, each total, the menus, the result, and the three counters at the end.
Explaining your code	-	No points of its own. As the syllabus says, a student who cannot explain the submitted work may receive a reduced grade.
Total	100	

12. Checklist before you hand in

- ☐ My program compiles with `gcc -Wall` without errors.
- ☐ Card points are right: A = 1; J, Q, K = 10.
- ☐ I play first, and I can choose Hit or Stand again and again.
- ☐ Above 21, the game ends at once, and the dealer does not play.
- ☐ The dealer plays only after I stand.
- ☐ The dealer draws below 17 and stops at 17 or more.
- ☐ Win, loss, dealer bust and draw all work. I tried each one.
- ☐ Typing 3 at a menu makes the program ask again.
- ☐ Play again starts a new game from 0; Quit prints wins, losses and draws.
- ☐ My program has the functions of section 6.4.
- ☐ The file is named `blackjack.c`.
- ☐ I can explain every line of my program.